



Plataforma de Gestión de Citas Médicas

(Clínica/Hospital)

Entrega de Avance 1

ASIGNATURA

Programación II

EQUIPO DE TRABAJO

Grupo 7

CATEDRÁTICO

Ing. Daniel Enrique Guevara Gomez

Integrantes del Equipo

11 MIEMBROS



MERINO
VENTURA
ALEJANDRO
SEBASTIAN

CIF: 2026020122

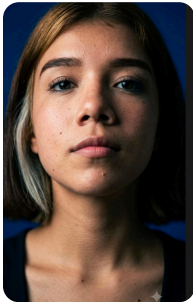
TRABAJÓ



MELGAR RIVAS
WILLIAM ARIEL

CIF: 2026011736

TRABAJÓ



FUENTES ORTIZ
ERIKA ALEXANDRA

CIF: 2026011709

TRABAJÓ



ORELLANA ROJAS
BAYRON
ALEXANDER

CIF: 2026011707

TRABAJÓ



VENTURA
VELASQUEZ
CARLOS MARIO

CIF: 2026011585

TRABAJÓ



FLORES
HERNANDEZ
WALTER
ALEJANDRO

CIF: 2026011012

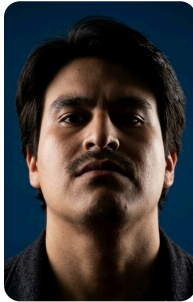
TRABAJÓ



SANCHEZ
MENJIVAR NICOLE
NOHEMY

CIF: 2026010813

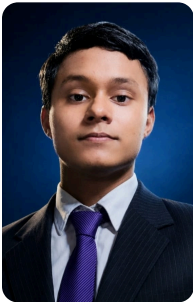
TRABAJÓ



DIAZ SANTOS ZAIR
BENETT

CIF: 2026010796

TRABAJÓ



VIGIL RAMIREZ
ALEJANDRO
ANTONIO

CIF: 2026010204

TRABAJÓ



LOPEZ RUIZ HECTOR
NAPOLEON

CIF: 2026010132

TRABAJÓ



VASQUEZ AMAYA
WALTER AMILCAR

CIF: 2026010068

TRABAJÓ



SECCIÓN 02

Objetivo General y Objetivos Específicos

Objetivo General

Desarrollar una plataforma web multiinquilino para la gestión de citas médicas en clínicas y hospitales, que permita a médicos, enfermeras, pacientes y administradores gestionar el ciclo completo de la atención — desde el registro del paciente y la programación de la cita hasta la generación de recetas y expedientes electrónicos — garantizando seguridad, trazabilidad y facilidad de uso para los distintos roles del sistema.

Objetivos Específicos

Se construyen progresivamente: 1 en Avance 1, 2 en Avance 2, 3 en entrega final.

01

Definir la arquitectura base, entidades del dominio y estructura del proyecto (backend Java Spring Boot + frontend React + base de datos PostgreSQL), aplicando la metodología Scrum para organizar el equipo y las Historias de Usuario iniciales del sistema.

02

Avance 2 — en construcción

03

Entrega Final — en construcción

SECCIÓN 03

Distribución del Equipo (Scrum)

Product Owner: LOPEZ RUIZ HECTOR NAPOLEON · Scrum Master: ORELLANA ROJAS BAYRON ALEXANDER

LIDERAZGO



LOPEZ RUIZ HECTOR NAPOLEON

PRODUCT OWNER

ANALISTA DE NEGOCIO / PROJECT MANAGER



ORELLANA ROJAS BAYRON ALEXANDER

SCRUM MASTER

DESARROLLADOR FULLSTACK



VENTURA VELASQUEZ CARLOS MARIO

ARCHITECT

ARQUITECTO DE SOFTWARE

DESARROLLO



VIGIL RAMIREZ ALEJANDRO ANTONIO

DEVELOPER

DESARROLLADOR BACKEND



FLORES HERNANDEZ WALTER ALEJANDRO

DEVELOPER

DESARROLLADOR BACKEND



DIAZ SANTOS ZAIR BENETT

DEVELOPER

DESARROLLADOR FRONTEND



MELGAR RIVAS WILLIAM ARIEL

DEVELOPER

DESARROLLADOR FRONTEND

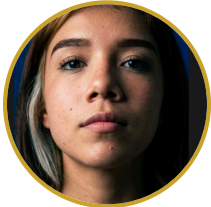


MERINO VENTURA ALEJANDRO SEBASTIAN

DEVELOPER

BASE DE DATOS

CALIDAD (QA)



FUENTES ORTIZ ERIKA ALEXANDRA

QA

ANALISTA DE PRUEBAS



VASQUEZ AMAYA WALTER AMILCAR

QA

ANALISTA DE PRUEBAS

NEGOCIO



SANCHEZ MENJIVAR NICOLE NOHEMY

BUSINESS ANALYST

ANALISTA DE NEGOCIO

SECCIÓN 04

Roles y Funciones del Sistema

Roles de usuario dentro del software, no roles Scrum.



Médico

Diagnóstico, toma de decisiones sobre el tratamiento y supervisión de la evolución del paciente.

- ✓ Diagnosticar
- ✓ Crear receta
- ✓ Ver expediente
- ✓ Crear expediente
- ✓ Modificar expediente
- ✓ Consultar historial de citas



Enfermera

Eje central del cuidado directo y continuo. Colabora con el médico.

- ✓ Programar cita
- ✓ Registrar paciente
- ✓ Completar datos del paciente
- ✓ Realizar clasificación (triaje)
- ✓ Consultar expediente



Administrador

Gestiona el sistema, usuarios y configuración del tenant.

- ✓ Administrar usuarios
- ✓ Configurar horarios de médicos
- ✓ Ver reportes y KPIs
- ✓ Auditoría de accesos y cambios



Recepcionista

Personal que gestiona la entrada de pacientes y el agendamiento de citas.

- ✓ Registrar pacientes nuevos
- ✓ Agendar citas según disponibilidad
- ✓ Confirmar asistencia
- ✓ Buscar paciente por nombre o CIF

SECCIÓN 05A

Requerimientos del Sistema

Historias de Usuario — Avance 1 y Avance 2.

PRIORIDAD	HU	HISTORIA	CRITERIOS DE ACEPTACIÓN
AVANCE 1 — IMPLEMENTADO			
ALTA	HU-007	Como usuario del sistema, quiero iniciar sesión de forma segura con mi rol y el código de mi clínica, para acceder únicamente a las funciones que me corresponden.	- Login con correo, contraseña y código de clínica - JWT emitido · Redirección según rol - Bloqueo tras 5 intentos en 15 minutos
ALTA	HU-008	Como administrador de clínica, quiero crear usuarios del sistema con su rol asignado, para controlar quién accede al sistema.	- Alta con nombre, correo, rol (DOCTOR / NURSE / RECEPTIONIST / ADMIN) - Contraseña temporal asignada - Usuario obliga a cambiar contraseña en primer login
AVANCE 2 — EN CONSTRUCCIÓN			
ALTA	HU-001	Como enfermera / recepcionista, quiero un formulario para registrar pacientes y consultar su expediente, para atenderlos rápidamente.	- Registro con nombre, DUI, fecha de nacimiento, contacto - Búsqueda por nombre o DUI - Expediente muestra historial de citas, diagnósticos y signos vitales
ALTA	HU-003	Como recepcionista, quiero agendar una cita para un paciente eligiendo médico y horario disponible, para gestionar la agenda sin doble reserva.	- Búsqueda de paciente por nombre o DUI - Selección de médico, especialidad, fecha y hora - Sistema impide doble reserva - Genera código de reserva único (ej. COD-0001)
MEDIA	HU-005	Como administrador, quiero gestionar los horarios de los médicos y la disponibilidad de consultorios, para optimizar la ocupación de la clínica.	- Crear, modificar y eliminar bloques horarios - Asignar consultorios a médicos - Visualizar ocupación en tiempo real
MEDIA	HU-009	Como médico, quiero consultar mi agenda del día en una pantalla, para prepararme antes de cada consulta.	- Lista ordenada por hora - Estado por cita (pendiente / en espera / atendida) - Un clic para abrir el expediente

SECCIÓN 05B

Requerimientos del Sistema

Historias de Usuario — Avance 3 (entrega final).

PRIORIDAD	HU	HISTORIA	CRITERIOS DE ACEPTACIÓN
AVANCE 3 — ENTREGA FINAL			
ALTA	HU-002	Como médico, quiero registrar una receta electrónica, para llevar el control del tratamiento del paciente.	- Receta almacenada en expediente - Asociada a paciente y médico - Incluye medicamentos, dosis y duración
MEDIA	HU-004	Como enfermera, quiero realizar el triaje de pacientes en espera y registrar sus signos vitales, para que el médico los reciba preparado.	- Registro de peso, talla, presión, temperatura, FC y síntomas - Asignación de prioridad (BAJO / MEDIO / ALTO / CRÍTICO) - Se guarda en el expediente clínico
BAJA	HU-006	Como recepcionista, quiero imprimir órdenes de atención para los pacientes, para entregarles un comprobante físico.	- PDF con datos de la cita y del médico - Incluye código de reserva
MEDIA	HU-010	Como administrador, quiero un dashboard con KPIs de la clínica, para tomar decisiones basadas en datos.	- KPIs en tiempo real (pacientes registrados, citas del mes, ocupación) - Filtro por período - Gráfico de tendencia

SECCIÓN 06

Alcances y Limitaciones

Alcances

Lo que la plataforma sí cubre en esta implementación.

- ✓ Gestión multiinquilino (múltiples clínicas en la misma plataforma).
- ✓ Registro y expediente electrónico de pacientes.
- ✓ Programación, reprogramación y cancelación de citas.
- ✓ Emisión de recetas asociadas a la consulta.
- ✓ Roles diferenciados: Médico, Enfermera, Recepcionista, Administrador.
- ✓ Autenticación segura con hashing BCrypt.
- ✓ Auditoría de acciones críticas.

Limitaciones

Fuera del alcance de este avance del proyecto.

- ✗ No integración con laboratorios externos.
- ✗ No facturación electrónica ni módulo contable completo.
- ✗ No telemedicina ni videoconsulta.
- ✗ No firma electrónica de recetas.
- ✗ No integración con sistemas de aseguradoras.
- ✗ No app móvil nativa (solo web responsive).

SECCIÓN 07A

Planificación

Avance 1 — Implementado.

#	ACTIVIDAD	RESPONSABLE	FECHA OBJETIVO	ESTADO
AVANCE 1 — IMPLEMENTADO				
1	Definir arquitectura y stack tecnológico	VIGIL RAMIREZ ALEJANDRO (Architect)	15/07/2026	Completado
2	Diseño del modelo de datos y schema.sql	VENTURA VELASQUEZ CARLOS (DBA)	22/07/2026	Completado
3	Configuración de Neon PostgreSQL	VENTURA VELASQUEZ CARLOS	25/07/2026	Completado
4	Estructura base backend (Spring Boot + JPA)	ORELLANA ROJAS BAYRON / FLORES HERNANDEZ WALTER	28/07/2026	Completado
5	Estructura base frontend (React + Vite + TS)	DIAZ SANTOS ZAIR / MELGAR RIVAS WILLIAM	30/07/2026	Completado
6	Sistema de logo MediSuite (SVG + componente)	LOPEZ RUIZ HECTOR	30/07/2026	Completado
7	Definición de Historias de Usuario iniciales	MERINO VENTURA ALEJANDRO	30/07/2026	Completado
8	Login multi-tenant con JWT (HU-007)	DIAZ SANTOS ZAIR / MELGAR RIVAS WILLIAM	01/08/2026	Completado
9	Seed de usuarios demo con BCrypt	MELGAR RIVAS WILLIAM	01/08/2026	Completado
10	Documentación técnica (TRD, PRD, APPFLOW)	VIGIL RAMIREZ ALEJANDRO	01/08/2026	Completado
11	Documento Avance 1	LOPEZ RUIZ HECTOR (PO)	02/08/2026	Completado
★	HITO — Entrega Avance 1	Todo el equipo	02/08/2026	Completado

SECCIÓN 07B

Planificación

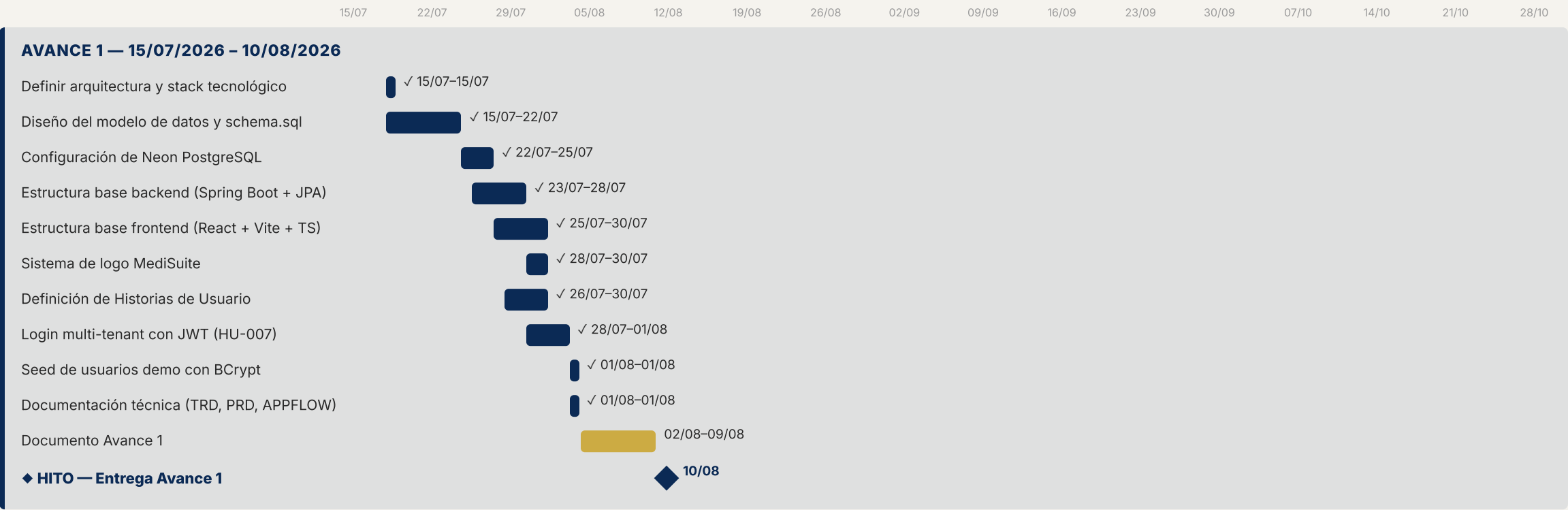
Avance 2 y Avance 3.

#	ACTIVIDAD	RESPONSABLE	FECHA OBJETIVO	ESTADO
AVANCE 2 — EN CONSTRUCCIÓN				
12	CRUD Pacientes — registro y búsqueda (HU-001)	MELGAR RIVAS / DIAZ SANTOS / ORELLANA / FLORES	25/08/2026	Pendiente
13	CRUD Citas y agenda médica (HU-003)	VASQUEZ AMAYA / VENTURA VELASQUEZ	10/09/2026	Pendiente
14	Módulo Expediente Electrónico	FUENTES ORTIZ / SANCHEZ MENJIVAR	18/09/2026	Pendiente
15	Módulo Recetas (HU-002)	MELGAR RIVAS WILLIAM / DIAZ SANTOS ZAIR	20/09/2026	Pendiente
16	Notificaciones por correo	VIGIL RAMIREZ / MERINO VENTURA	22/09/2026	Pendiente
17	QA Avance 2	FUENTES ORTIZ ERIKA / SANCHEZ MENJIVAR NICOLE	25/09/2026	Pendiente
18	Documento Avance 2	LOPEZ RUIZ HECTOR (PO)	26/09/2026	Pendiente
★	HITO — Entrega Avance 2	Todo el equipo	26/09/2026	Pendiente
AVANCE 3 — ENTREGA FINAL				
19	Dashboard y reportes de citas (HU-010)	VIGIL RAMIREZ / VASQUEZ AMAYA	12/10/2026	Pendiente
20	Gestión de usuarios y roles (HU-008)	MERINO VENTURA / ORELLANA ROJAS	15/10/2026	Pendiente
21	QA integral y correcciones finales	FUENTES ORTIZ ERIKA / SANCHEZ MENJIVAR NICOLE	26/10/2026	Pendiente
22	Preparación defensa y documento final	Todo el equipo	29/10/2026	Pendiente
★	HITO — Entrega Final y Defensa	Todo el equipo	31/10/2026	Pendiente

SECCIÓN 08A

Cronograma Gantt

Detalle del Avance 1 (15/07 – 10/08/2026).

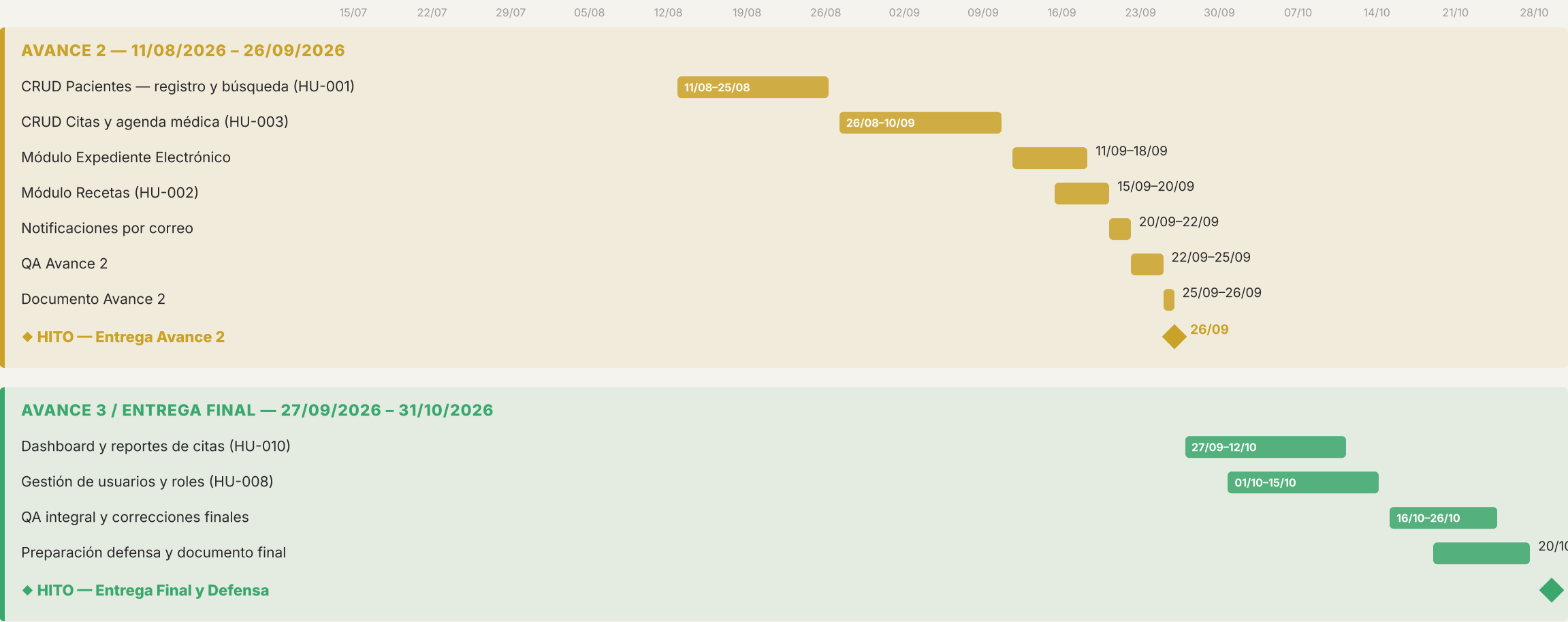


Avance 1Avance 2Avance 3Hito de entregaCompletado

SECCIÓN 08B

Cronograma Gantt

Avance 2 y Avance 3 / Entrega final (11/08 – 31/10/2026).



SECCIÓN 09.1A

Entradas / Salidas del Sistema

HU-001 — Avance 2 · HU-002 — Avance 3

HU-001 — Registro y consulta de pacientes

ENTRADAS

- Nombre completo
- DUI
- Fecha de nacimiento
- Teléfono
- Dirección
- Contacto de emergencia
- Tipo de sangre
- Alergias conocidas

SALIDAS

- Perfil del paciente registrado y confirmación de guardado
- Pantalla de expediente con historial de citas y diagnósticos
- Resultado de búsqueda por nombre o DUI

EVIDENCIA — BASE DE DATOS NEON

1 ✓
2
3

```
SELECT id, first_name, last_name, cif, birth_date, blood_type
FROM patients
WHERE tenant_id = 1;
```

Connected (1 query)

Run

Explain

Analyze

123ms

2 rows

#	id	first_name	last_name	cif	birth_date	blood_type
1	1	Maria	Lopez	04141199009999	1990-05-12	O+
2	2	Pedro	Hernandez	04141198512345	1985-11-03	A-

HU-002 — Receta electrónica

ENTRADAS

Medicamento, dosis, unidad, duración, indicaciones, ID médico, ID paciente

SALIDAS

Receta guardada en expediente, fecha/hora de emisión, lista cronológica

Entradas / Salidas del Sistema

Avance 2 — HU-003.

HU-003 — Agendamiento de cita

ENTRADAS

- Búsqueda de paciente por nombre o DUI
- Médico seleccionado
- Especialidad
- Fecha y hora disponible
- Motivo de consulta

SALIDAS

- Código de reserva único (ej. COD-0001)
- Cita registrada con estado SCHEDULED
- Agenda del médico actualizada

EVIDENCIA — BASE DE DATOS NEON (TABLA APPOINTMENTS)

1

2

3

4

5

SELECT column_name, data_type

FROM information_schema.columns

WHERE table_name = 'appointments'

ORDER BY ordinal_position;

Connected (no queries)

Run

Explain

Analyze

#	column_name	data_type
1	id	bigint
2	tenant_id	bigint
3	patient_id	bigint
4	doctor_id	bigint
5	scheduled_at	timestamp with time zone
6	status	character varying
7	reason	character varying
8	office	character varying
9	reservation_code	character varying
10	created_at	timestamp with time zone
11	updated_at	timestamp with time zone

SECCIÓN 09.1B

Entradas / Salidas del Sistema

HU-004 — Avance 3 · HU-008 — Avance 1

HU-004 — Triage y signos vitales

ENTRADAS

- Búsqueda por DUI o nombre
- Peso (kg)
- Talla (cm)
- Presión arterial (mmHg)
- Temperatura (°C)
- Frecuencia cardíaca (bpm)
- Síntomas
- Nivel de prioridad

SALIDAS

- Signos vitales guardados en expediente
- Paciente clasificado con prioridad (BAJO / MEDIO / ALTO / CRÍTICO)
- Historial visible para el médico

HU-008 — Alta de usuarios

ENTRADAS

- Nombre completo
- Correo electrónico
- Rol asignado (DOCTOR / NURSE / RECEPTIONIST / ADMIN)
- Clínica asociada

SALIDAS

- Usuario creado y activo
- Contraseña temporal asignada
- Confirmación de registro al administrador

EVIDENCIA — BASE DE DATOS NEON

1 SELECT id, first_name, last_name, email, role, active

2 FROM users

3 WHERE tenant_id = 1;

4 |

Connected (1 query)

Run

Explain

Analyze

1s

4 rows

#	id	first_name	last_name	email	role	active
1	1	Ana	Martínez	ana.martinez.demo@medisuite.test	DOCTOR	t
2	3	Beatriz	Reyes	beatriz.reyes.demo@medisuite.test	ADMIN	t
3	2	Carlos	Gómez	carlos.gomez.demo@medisuite.test	NURSE	t
4	4	Jorge	Alas	jorge.alas.demo@medisuite.test	RECEPTIONIST	t

SECCIÓN 09.2

Entradas / Salidas del Sistema

HU-007 — Login multi-tenant (caso ya funcional).

HU-007 — Login multi-tenant

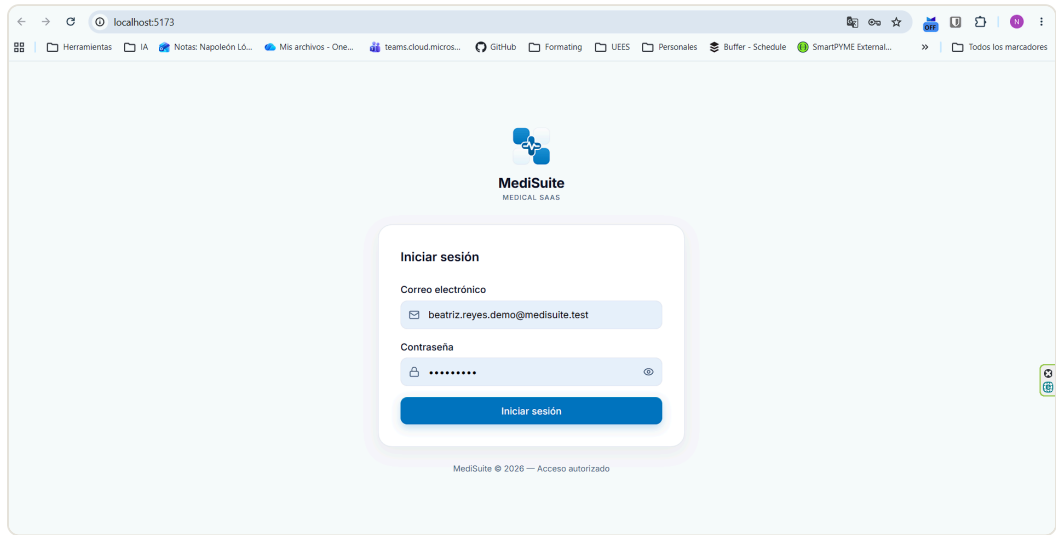
ENTRADAS

- Código de tenant
- Correo
- Contraseña

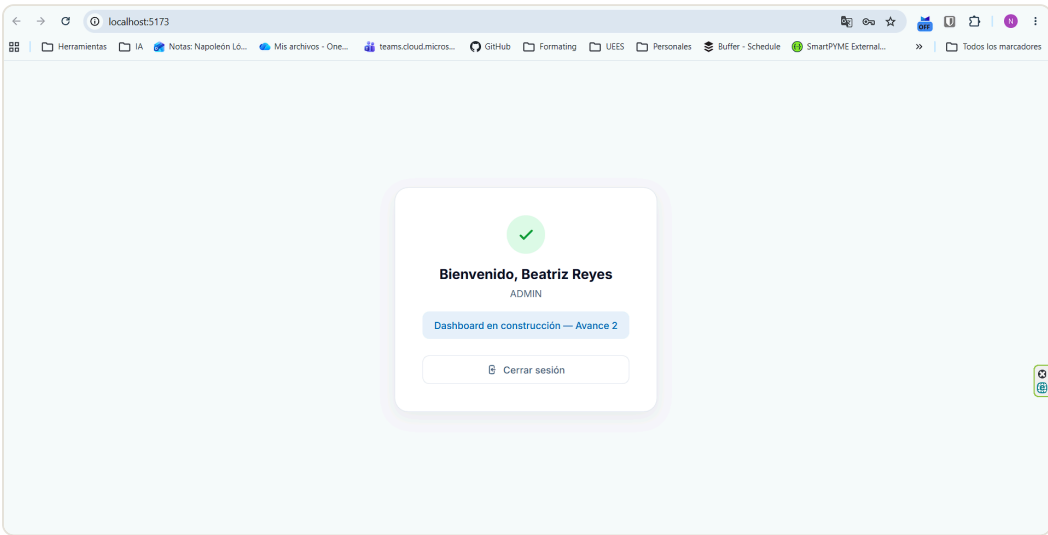
SALIDAS

- Token JWT
- Redirección a dashboard
- Mensaje "Bienvenido {nombre}"

PANTALLA DE LOGIN



LOGIN EXITOSO — SALIDA DEL SISTEMA



RESPUESTA DEL API (JWT)

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Cargar los perfiles personales y de sistema tardó 2288ms.
(base) PS C:\Users\hlopez> Invoke-RestMethod -Method POST -Uri "http://localhost:8080/api/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"tenantSlug":"clinica-san-rafael","email":"beatriz.reyes.demo@medisuite.test","password":"Demo2026!"}'

accessToken
-----
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIzIiwicm9sZSI6IkkFETU1OiwidGVuYW50X2lkIjoxLCJpYXQiOiJlE3ODU3ODE0NDUsImV4cCI6MTc4NTc4MjM0NX0.UG0dBJMILcPaW364HEhH3Wx_3maGGbv...

(base) PS C:\Users\hlopez> |
```



SECCIÓN 09.3

Prueba con cuentas demo — HU-007

Validación de login exitoso para cada rol del sistema.

Bienvenido, Ana Martínez

DOCTOR

Dashboard en construcción — Avance 2

Cerrar sesión

DOCTOR — Ana Martínez

Bienvenido, Carlos Gómez

NURSE

Dashboard en construcción — Avance 2

Cerrar sesión

NURSE — Carlos Gómez

Bienvenido, Jorge Alas

RECEPTIONIST

Dashboard en construcción — Avance 2

Cerrar sesión

RECEPTIONIST — Jorge Alas

Bienvenido, Beatriz Reyes

ADMIN

Dashboard en construcción — Avance 2

Cerrar sesión

ADMIN — Beatriz Reyes

SECCIÓN 10

Declaración de Entidades

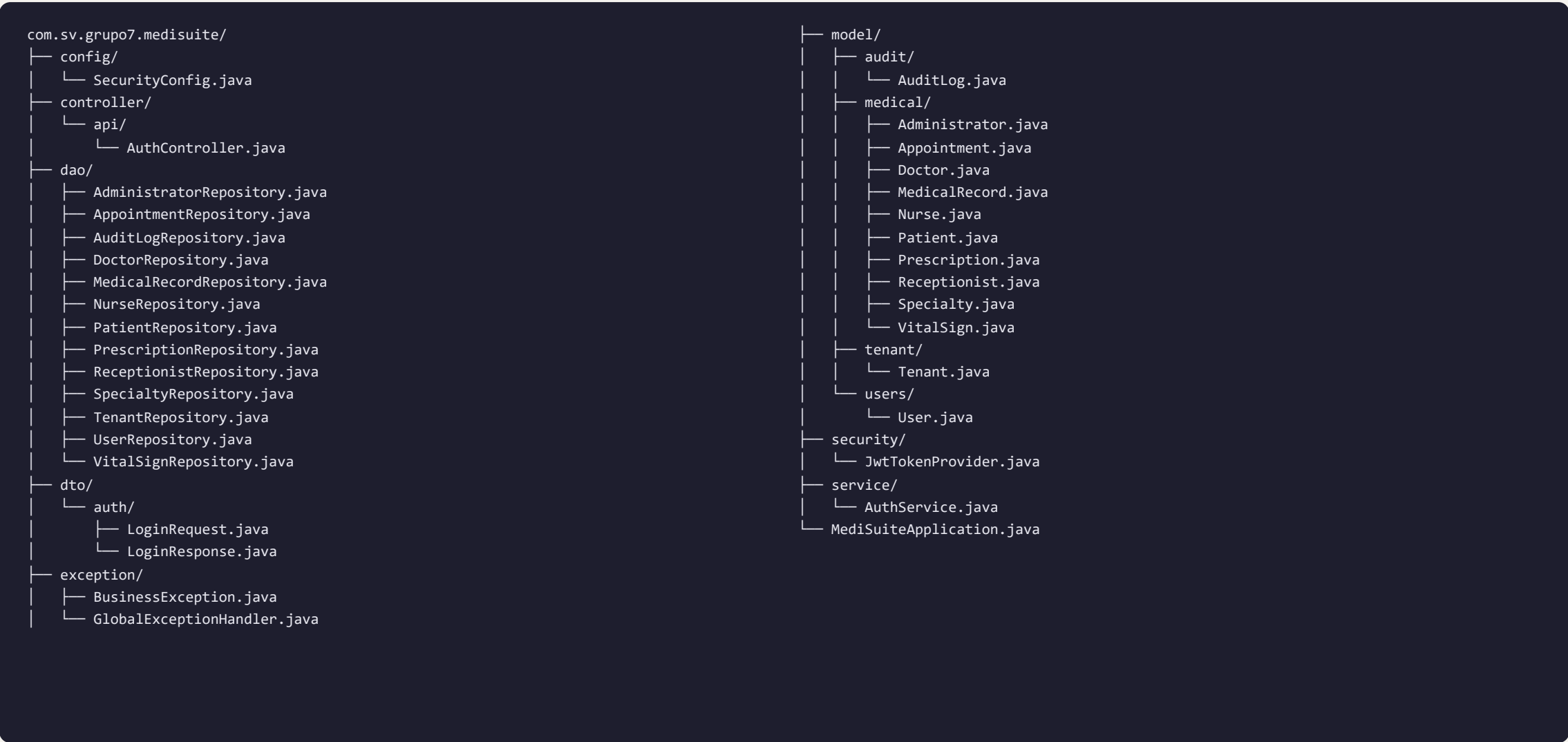
Modelo de dominio según requerimientos.

ENTIDAD	DESCRIPCIÓN	ATRIBUTOS
■ tenants	Clínica u hospital cliente de la plataforma.	id, slug, name, plan, is_active, created_at
■ users	Cuenta con acceso al sistema (rol vía ENUM).	id, tenant_id, email, password_hash, first_name, last_name, role, active, created_at
■ administrators	Administrador con nivel de permisos asignado.	id, tenant_id, user_id, permission_level, created_at
■ doctors	Perfil profesional de un médico.	id, tenant_id, user_id, specialty, license_number, created_at
■ nurses	Perfil de enfermera con turno asignado.	id, tenant_id, user_id, shift, assigned_area, created_at
■ receptionists	Perfil de recepcionista con turno y consultorio.	id, tenant_id, user_id, shift, assigned_office, created_at
■ specialties	Especialidades médicas disponibles por clínica.	id, tenant_id, name, active
■ patients	Persona que recibe atención (entidad de datos).	id, tenant_id, first_name, last_name, cif, birth_date, phone, blood_type, allergies
■ appointments	Reserva de consulta médica.	id, tenant_id, patient_id, doctor_id, scheduled_at, status, reason, reservation_code, office, created_at
■ medical_records	Expediente clínico del paciente.	id, tenant_id, patient_id, created_at
■ vital_signs	Signos vitales y triaje registrados.	id, medical_record_id, appointment_id, weight_kg, height_cm, blood_pressure, temperature_c, heart_rate, symptoms, priority, recorded_at
■ prescriptions	Recetas emitidas por el médico.	id, tenant_id, medical_record_id, doctor_id, medications, dosage, duration, instructions
■ audit_logs	Registro de auditoría de acciones críticas.	id, tenant_id, user_id, action, entity_name, entity_id, data_before, data_after, ip_address, created_at

SECCIÓN 11

Proyecto Base — Estructura

Paquete principal: com.sv.grupo7.medisuite



Stack Backend

 Java 21  Spring Boot 3.4
Spring Data JPA · Spring Security · JWT · BCrypt  Maven

Stack Frontend

 React 19  Vite  TypeScript  Tailwind CSS 4
React Router · pnpm

Base de Datos

 PostgreSQL 16 (Neon) 13 tablas · schema.sql versionado  Flyway

Clases por paquete

model/medical/	→ Appointment.java, Doctor.java, MedicalRecord.java, Nurse.java, Patient.java, Prescription.java, Receptionist.java, Administrator.java, VitalSign.java
model/users/	→ User.java
model/tenant/	→ Tenant.java
model/audit/	→ AuditLog.java
dao/	→ Un Repository por cada entidad (13 en total)
security/	→ JwtTokenProvider.java
service/	→ AuthService.java
dto/auth/	→ LoginRequest.java, LoginResponse.java



SECCIÓN 11B

Proyecto Base — Entidades Principales

Clases JPA mapeadas a tablas de Neon PostgreSQL.

USER.JAVA

```
package com.sv.grupo7.medisuite.model.users;

@Entity
@Table(name = "users")
@Getter @Setter
public class User {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private Tenant tenant;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(nullable = false)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @Column(nullable = false)
    private String role; // DOCTOR | NURSE | RECEPTIONIST | ADMIN

    @Column(nullable = false)
    private Boolean active;

    @Column(name = "failed_login_attempts")
    private Integer failedLoginAttempts;

    @Column(name = "locked_until")
    private OffsetDateTime lockedUntil;
}
```

PATIENT.JAVA

```
package com.sv.grupo7.medisuite.model.medical;

@Entity
@Table(name = "patients")
@Getter @Setter
public class Patient {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private Tenant tenant;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(nullable = false)
    private String cif;

    @Column(name = "birth_date", nullable = false)
    private LocalDate birthDate;

    @Column(name = "blood_type")
    private String bloodType;

    @Column(columnDefinition = "text")
    private String allergies;

    @PrePersist
    void prePersist() {
        sc-camel-created-at = sc-camel-updated-at = OffsetDateTime.now();
    }
}
```

SECCIÓN 11C

Proyecto Base — Servicio y Acceso a Datos

Capa de autenticación: repositorio JPA y lógica de negocio.

USERREPOSITORY.JAVA

```
package com.sv.grupo7.medisuite.dao;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmailAndTenantId(String email, Long tenantId);
}
```

AUTHSERVICE.JAVA

```
package com.sv.grupo7.medisuite.service;

@Service
@RequiredArgsConstructor
public class AuthService {

    private static final int MAX_ATTEMPTS = 5;
    private static final int LOCK_MINUTES = 15;

    private final UserRepository userRepository;
    private final TenantRepository tenantRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtTokenProvider jwtTokenProvider;

    @Transactional
    public LoginResponse login(LoginRequest req) {
        Tenant tenant = tenantRepository.findBySlug(req.tenantSlug())
            .orElseThrow(() -> new BadCredentialsException("Credenciales inválidas"));

        User user = userRepository.findByEmailAndTenantId(req.email(), tenant.getId())
            .orElseThrow(() -> new BadCredentialsException("Credenciales inválidas"));

        if (Boolean.FALSE.equals(user.getActive()))
            throw new BusinessException("Cuenta inactiva.");

        if (user.getLockedUntil() != null && user.getLockedUntil().isAfter(OffsetDateTime.now()))
            throw new BusinessException("Cuenta bloqueada temporalmente.");

        if (!passwordEncoder.matches(req.password(), user.getPasswordHash())) {
            registerFailedAttempt(user);
            throw new BadCredentialsException("Credenciales inválidas");
        }

        user.setFailedLoginAttempts(0);
        user.setLockedUntil(null);

        String token = jwtTokenProvider.generateToken(
            user.getId(), user.getRole(), user.getTenantId());

        return new LoginResponse(token, "Bearer",
            jwtTokenProvider.getExpirationSeconds(),
            new LoginResponse.UserInfo(
                user.getId(), user.getFirstName() + " " + user.getLastName(),
                user.getRole(), user.getTenantId()));
    }

    private void registerFailedAttempt(User user) {
        int attempts = user.getFailedLoginAttempts() + 1;
        user.setFailedLoginAttempts(attempts);
        if (attempts >= MAX_ATTEMPTS)
            user.setLockedUntil(OffsetDateTime.now().plusMinutes(LOCK_MINUTES));
    }
}
```

SECCIÓN 11D

Proyecto Base — Entidades de Cita y Médico

Relaciones JPA entre las entidades principales del dominio clínico.

APPOINTMENT.JAVA

```
package com.sv.grupo7.medisuite.model.medical;

@Entity
@Table(name = "appointments")
@Getter @Setter
public class Appointment {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private Tenant tenant;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "patient_id", nullable = false)
    private Patient patient;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "doctor_id", nullable = false)
    private Doctor doctor;

    @Column(name = "scheduled_at", nullable = false)
    private OffsetDateTime scheduledAt;

    @Column(nullable = false, length = 20)
    private String status;

    @Column(length = 200)
    private String reason;

    @Column(length = 50)
    private String office;

    @Column(name = "reservation_code", nullable = false, length = 10)
    private String reservationCode;

    @PrePersist
    void prePersist() {
        sc-camel-created-at = sc-camel-updated-at = OffsetDateTime.now();
    }
}
```

DOCTOR.JAVA

```
package com.sv.grupo7.medisuite.model.medical;

@Entity
@Table(name = "doctors")
@Getter @Setter
public class Doctor {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tenant_id", nullable = false)
    private Tenant tenant;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false, unique = true)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "specialty_id", nullable = false)
    private Specialty specialty;

    @Column(name = "license_number", nullable = false, length = 30)
    private String licenseNumber;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(name = "available_schedule", sc-camel-column-definition = "jsonb")
    private String availableSchedule;

    @PrePersist
    void prePersist() {
        sc-camel-created-at = sc-camel-updated-at = OffsetDateTime.now();
    }
}
```

SECCIÓN 11E

Proyecto Base — Evidencia del Sistema

Sistema compilando y ejecutándose en entorno de desarrollo.

COMPILACIÓN BACKEND

```
Windows PowerShell

poration. All rights reserved.

les y de sistema tardó 2460ms.
cd C:\Users\hlopez\medisuite
edisuite>
edisuite> $env:DB_HOST="ep-nameless-water-avvbs1vv-pooler.c-11.us-east-1.aws.neon.tech"
edisuite> $env:DB_PORT="5432"
edisuite> $env:DB_NAME="clinica_dev"
edisuite> $env:DB_USERNAME="clinica_app"
edisuite> $env:DB_PASSWORD="clinica_dev_2026"
edisuite> $env:DB_SSLMODE="require"
edisuite> $env:JWT_SECRET="medisuite-dev-secret-key-32chars-ok!!"
edisuite> $env:JWT_EXPIRATION_MS="900000"
edisuite> $env:APP_PORT="8080"
edisuite> $env:APP_ENV="development"
edisuite> cd backend
edisuite\backend> mvn spring-boot:run
S...

com.sv.grupo:medisuite-backend >-----
ackend 0.0.1-SNAPSHOT

-----[ jar ]-----

2:run (default-cli) > test-compile @ medisuite-backend >>>

resources (default-resources) @ medisuite-backend ---
rom src\main\resources to target\classes
rom src\main\resources to target\classes

compile (default-compile) @ medisuite-backend ---
all classes are up to date.

testResources (default-testResources) @ medisuite-backend ---
sourceDirectory C:\Users\hlopez\medisuite\backend\src\test\resources

testCompile (default-testCompile) @ medisuite-backend ---
e
```

BACKEND CORRIENDO (SPRING BOOT)

```
Windows PowerShell

00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.HikariDataSource      : HikariPool-1
00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.pool.HikariPool      : HikariPool-1
00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.HikariDataSource      : HikariPool-1
00 WARN 18748 --- [medisuite] [      main] org.hibernate.orm.deprecation          : HHH90000025:
licitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
00 INFO 18748 --- [medisuite] [      main] o.h.e.t.j.p.i.JtaPlatformInitiator      : HHH000489: N
jta.platform' to enable JTA platform integration)
00 INFO 18748 --- [medisuite] [      main] j.LocalContainerEntityManagerFactoryBean : Initialized
00 WARN 18748 --- [medisuite] [      main] .s.s.UserDetailsServiceAutoConfiguration :

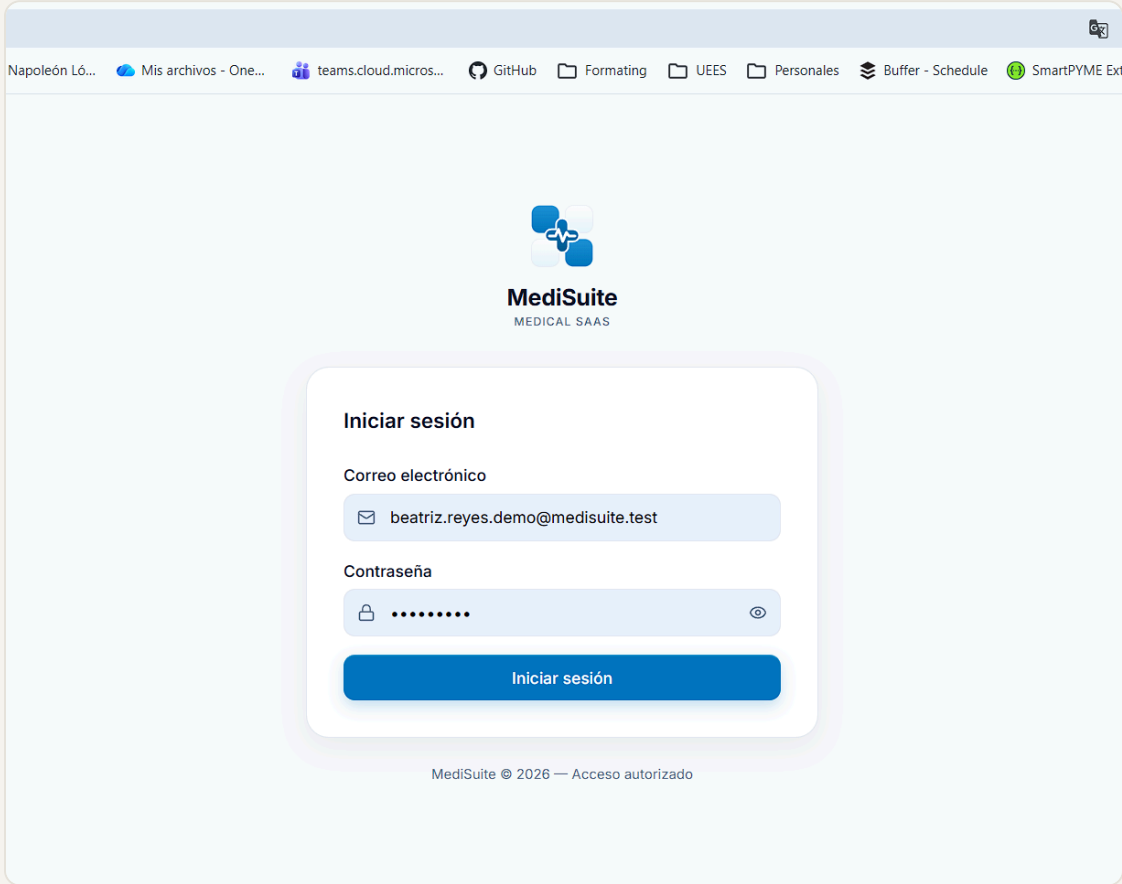
password: 4a053bc9-d7e4-4fa3-8cc1-3fe4e80b0e43

For development use only. Your security configuration must be updated before running your application in

00 INFO 18748 --- [medisuite] [      main] r$InitializeUserDetailsManagerConfigurer : Global Auth
bean with name inMemoryUserDetailsManager
00 INFO 18748 --- [medisuite] [      main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 2 e
00 INFO 18748 --- [medisuite] [      main] o.s.d.j.r.query.QueryEnhancerFactory      : Hibernate is
00 INFO 18748 --- [medisuite] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer   : Tomcat start
00 INFO 18748 --- [medisuite] [      main] c.s.g.h.citas.MediSuiteApplication        : Started Medi
or 7.449)
00 INFO 18748 --- [medisuite] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing
00 INFO 18748 --- [medisuite] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing
00 INFO 18748 --- [medisuite] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed in
00 DEBUG 18748 --- [medisuite] [nio-8080-exec-2] org.hibernate.SQL                        :

e,
ge,
```

FRONTEND CORRIENDO (VITE + REACT)



CONEXIÓN A BD ACTIVA

```

=====
:: Spring Boot ::
(v3.3.2)

2026-08-03T12:15:25.728-06:00 INFO 18748 --- [medisuite] [      main] c.s.g.h.citas.MediSuiteApplication        : Starting MediSuiteApplication using J
ava 21.0.11 with PID 18748 (C:\Users\hlopez\medisuite\backend\target\classes started by hlopez in C:\Users\hlopez\medisuite\backend)
2026-08-03T12:15:25.730-06:00 INFO 18748 --- [medisuite] [      main] c.s.g.h.citas.MediSuiteApplication        : The following 1 profile is active: "d
evelopment"
2026-08-03T12:15:26.474-06:00 INFO 18748 --- [medisuite] [      main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA reposi
ories in DEFAULT mode.
2026-08-03T12:15:26.553-06:00 INFO 18748 --- [medisuite] [      main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scann
ing in 70 ms. Found 17 JPA repository interfaces.
2026-08-03T12:15:27.201-06:00 INFO 18748 --- [medisuite] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer   : Tomcat initialized with port 8080 (ht
tp)
2026-08-03T12:15:27.226-06:00 INFO 18748 --- [medisuite] [      main] o.apache.catalina.core.StandardService    : Starting service [Tomcat]
2026-08-03T12:15:27.226-06:00 INFO 18748 --- [medisuite] [      main] o.apache.catalina.core.StandardEngine     : Starting Servlet engine: [Apache Tomc
at/10.1.26]
2026-08-03T12:15:27.309-06:00 INFO 18748 --- [medisuite] [      main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebAppli
cationContext
2026-08-03T12:15:27.310-06:00 INFO 18748 --- [medisuite] [      main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initializ
ation completed in 1545 ms
2026-08-03T12:15:27.565-06:00 INFO 18748 --- [medisuite] [      main] org.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnit
Info [name: default]
2026-08-03T12:15:27.640-06:00 INFO 18748 --- [medisuite] [      main] o.hibernate.Version                      : HHH0000412: Hibernate ORM core version
6.5.2.Final
2026-08-03T12:15:27.673-06:00 INFO 18748 --- [medisuite] [      main] o.h.c.internal.RegionFactoryInitiator     : HHH000026: Second-level cache disable
d
2026-08-03T12:15:27.948-06:00 INFO 18748 --- [medisuite] [      main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA
class transformer
2026-08-03T12:15:27.971-06:00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.HikariDataSource        : HikariPool-1 - Starting...
2026-08-03T12:15:29.165-06:00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection org.p
ostgresql.jdbc.PgConnection@37dec463
2026-08-03T12:15:29.167-06:00 INFO 18748 --- [medisuite] [      main] com.zaxxer.hikari.HikariDataSource        : HikariPool-1 - Start completed.
2026-08-03T12:15:29.330-06:00 WARN 18748 --- [medisuite] [      main] org.hibernate.orm.deprecation            : HHH90000025: PostgreSQLDialect does n
```



SECCIÓN 12

Conclusiones

Reflexiones individuales sobre las actividades desarrolladas en el Avance 1.

LÓPEZ RUIZ HÉCTOR NAPOLEON

Durante el Avance 1 lideré la definición de la arquitectura multi-tenant de MediSuite, estableciendo las bases técnicas con Spring Boot 3 y PostgreSQL en Neon. Las decisiones de diseño tomadas en esta etapa tienen impacto directo en la mantenibilidad del sistema. Aprendí que una arquitectura bien planificada desde el inicio reduce significativamente los problemas en fases posteriores del proyecto.

ORELLANA ROJAS BAYRON ALEXANDER

Como Scrum Master facilité las ceremonias ágiles y coordiné las tareas del sprint del Avance 1. Participé también en el desarrollo fullstack conectando la lógica del backend con los componentes del frontend. Esta experiencia me permitió comprender la importancia de la comunicación entre capas del sistema y el rol del liderazgo técnico en un equipo distribuido.

VIGIL RAMÍREZ ALEJANDRO ANTONIO

En el Avance 1 implementé el endpoint de autenticación con Spring Security y JWT, garantizando la seguridad del acceso por tenant. La integración de BCrypt y la lógica de bloqueo por intentos fallidos fueron los aspectos más desafiantes. Comprendí la importancia de implementar seguridad desde las primeras etapas del desarrollo.

DÍAZ SANTOS ZAIR BENETT

Desarrollé la página de login de MediSuite utilizando React 19, TypeScript y Tailwind CSS, implementando la autenticación multi-tenant. La gestión del estado con contextos de React y la comunicación con el API REST fueron los principales retos técnicos. Este avance me permitió consolidar mis conocimientos en desarrollo frontend moderno.

FLORES HERNÁNDEZ WALTER ALEJANDRO

En el Avance 1 participé en la documentación del sistema y en el desarrollo backend con Spring Boot. La elaboración de las historias de usuario y la definición de criterios de aceptación me ayudaron a comprender mejor los requerimientos. Aprendí la importancia de mantener la documentación técnica alineada con la implementación real.

MELGAR RIVAS WILLIAM ARIEL

Contribuí al desarrollo del frontend de MediSuite con React y Vite, trabajando en los componentes de la interfaz de usuario. La migración de npm a pnpm y la configuración del entorno de desarrollo me permitieron mejorar mis habilidades en gestión de dependencias. Comprendí la relevancia de mantener un entorno estandarizado para todo el equipo.

MERINO VENTURA ALEJANDRO SEBASTIÁN

Mi principal aporte fue el diseño e implementación del esquema de base de datos con 13 tablas en Neon PostgreSQL. La definición de relaciones entre entidades y la integridad referencial fueron los aspectos más complejos. Este trabajo me permitió profundizar en el diseño de bases de datos relacionales y en el uso de PostgreSQL serverless.

FUENTES ORTIZ ERIKA ALEXANDRA

En el Avance 1 diseñé los casos de prueba para HU-007 y HU-008, validando el login multi-tenant y el alta de usuarios. La verificación del flujo de autenticación con diferentes roles me permitió identificar escenarios límite importantes. Comprendí que la calidad del software se garantiza desde las primeras etapas y no únicamente al final.

VÁSQUEZ AMAYA WALTER AMÍLCAR

Colaboré en las pruebas de integración del backend, verificando que los endpoints respondieran según los criterios de aceptación definidos. El análisis de los logs de Spring Boot y las respuestas del API fueron fundamentales para identificar defectos. Esta experiencia reforzó mi comprensión sobre la importancia de las pruebas en proyectos ágiles.

VENTURA VELÁSQUEZ CARLOS MARIO

Implementé las entidades JPA del dominio médico, incluyendo Doctor, Patient, Appointment y sus repositorios. La configuración de relaciones @ManyToOne y @OneToOne con lazy loading fue uno de los aspectos más técnicos. Aprendí que un buen modelo de datos es la base de cualquier sistema de información confiable.

SÁNCHEZ MENJÍVAR NICOLE NOHEMY

Me enfoqué en el levantamiento y documentación de los requerimientos funcionales, redactando las historias de usuario con sus criterios de aceptación. La identificación de actores y la definición de flujos de negocio para cada rol fueron los aspectos más complejos. Este trabajo me permitió comprender la importancia del análisis de requerimientos como base del desarrollo de software.



SECCIÓN 13

Bibliografía

Fuentes consultadas — Formato APA 7ª edición.

1. VMware, Inc. (2024). *Spring Boot Reference Documentation* (v3.3.2). <https://docs.spring.io/spring-boot/docs/3.3.2/reference/html/>
2. VMware, Inc. (2024). *Spring Security Reference Documentation* (v6.x). <https://docs.spring.io/spring-security/reference/>
3. Neon Technologies, Inc. (2024). *Neon Serverless PostgreSQL Documentation*. <https://neon.tech/docs>
4. Meta Open Source. (2024). *React Documentation* (v19). <https://react.dev/>
5. Vercel. (2024). *Vite Build Tool Documentation* (v5.x). <https://vitejs.dev/>
6. Tailwind Labs. (2024). *Tailwind CSS Documentation* (v4). <https://tailwindcss.com/docs>
7. Auth0 by Okta. (2024). *JSON Web Tokens Introduction*. <https://jwt.io/introduction>
8. OpenJDK. (2024). *OpenJDK 21 Documentation*. <https://openjdk.org/projects/jdk/21/>
9. Red Hat, Inc. (2024). *Hibernate ORM 6.5 Documentation*. <https://hibernate.org/orm/documentation/6.5/>
10. The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
11. Walls, C. (2022). *Spring Boot in Action* (2ª ed.). Manning Publications.